Principal Component Analysis

Intro and Motivation

PCA is the most widely used technique for dimensionality reduction in machine learning. It addresses a fundamental challenge: real datasets often have many features, but the intrinsic structure of the data may live in a lower dimensional subspace. PCA finds this subspace - the directions in which the data varies most - and projects the data onto it, discarding directions that contribute little information.

There are three reasons we want to reduce dimensionality.

1.) The curse of dimensionality:
as the number of features grows, the volume of the space grows exponentially, making data sparse and distances meaningless
Models trained in high-dimensional spaces with limited data overfit badly.

2. Computational Efficiency:
lower dimensional representations are cheaper to store, transmit and process

3. Visualisation
Humans can only visualise 2 or 3 dimensions, so projecting high dimensional data to 2D or 3D via PCA is a standard exploratory tool.

PCA is an unsupervised method.

The Setup

We have n data points in d dimensions, stacked as rows of the data matrix $X \in \mathbb{R}^{(n \times d)}$. Each row is a d-dimensional observation. PCA finds a set of k orthogonal dimensions ($k \leq d$) along which the data varies most, and projects onto these dimensions..

Since variance is measured around the mean, we must first center the data. We subtract the mean of each feature from all the input vectors. Features are along the columns:

$$\bar{x}_j = \frac{1}{n} \sum_i x_{ij} \quad \text{for feature } j$$

$$X_{\text{centered}} = X - 1\bar{x}^T$$

where 1 is a vector of ones, and $\bar{x}$ is the mean vector. Thus $1\bar{x}^T$ is a square matrix where each row is a duplicate of all the feature means. Going forward, we simply assume the data is centered, and just refer to it as X.

We then find the covariance matrix (Recall, it tells you how features i and j vary with each other).

$$C = \frac{1}{n-1} X^T X \in \mathbb{R}^{d \times d}$$

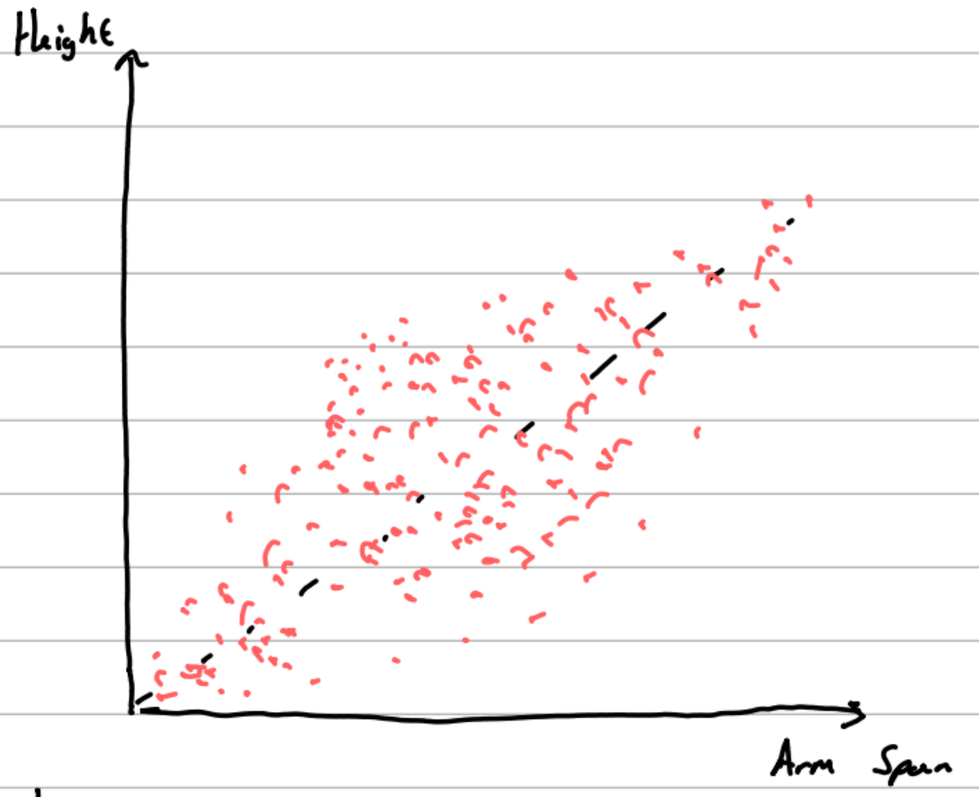
Thus $C_{ii} = \text{Var}(x_i)$, the diagonals are the variances. Thus, the total variance:

$$\text{Total Variance} = \text{tr}(C) = \sum_j \text{Var}(x_j)$$

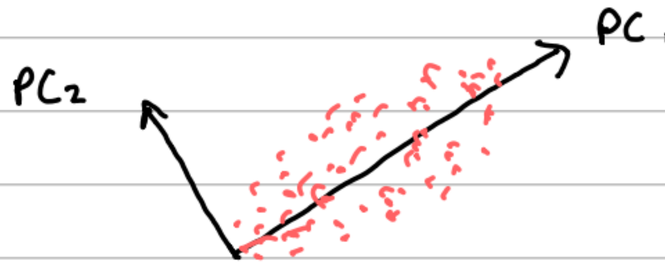
PCA redistributes the total variance across the principal components. The first component captures as much variance as possible, the second captures as much as possible of the remainder, and so on.

Why are we allowed to do this? Well, the total variance is a property of the data itself, it cannot be

created or destroyed. Thus, it is completely unaffected by any linear transformation. Consider two features that are highly correlated, like height (x_1) and arm span (x_2). Both have high variance individually, but because they move together most of the data lies along a single diagonal direction.



The axes do not align with the actual spread of data. PCA rotates the axes so that each axis captures as much of the variance as possible.



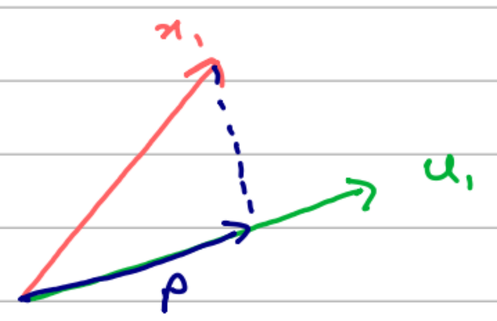
So now tall people with long arms are on one end, short people with short arms are the other. That gives us one more useful number than the two original axes.

To give further Geometric Intuition, the Covariance matrix encodes the shape of the data cloud. A diagonal C means features are uncorrelated, the data cloud is axis-aligned. A full C with large off-diagonal entries means features are correlated - the data cloud is an elongated ellipsoid tilted in some direction. PCA finds the axes of this ellipsoid, the directions which it is longest (most variance) and shortest (least variance). These are the eigenvectors of C. The eigenvalues tell you the length of each axis, how much variance lies along each direction.

Derivation: Variance Maximisation

We want to find the direction in the original space with the most variance. Let's call this direction $u_1 \in \mathbb{R}^d$.

Take one point x_1.



We project x_1 onto u_1 giving a projection vector

$$p_1 = \alpha_1 u_1$$

We want to find the coordinate α_1, which tells us how far along u_1 we are. See that $x_1 - p$ must be perpendicular to u_1:

$$u_1^T (x - p) = 0$$

$$u_1^T (x - \alpha_1 u_1) = 0$$

$$u_1^T x_1 - \alpha_1 u_1^T u_1 = 0$$

Since u_1 is a unit vector, $u_1^T u_1 = 1$

$$u_1^T x_1 - \alpha_1 = 0$$

$$\alpha_1 = u_1^T x_1$$

And similarly for all other points

$$\alpha_i = x_i^T u_1$$

$$\Rightarrow \quad X u_1 = \begin{bmatrix} \alpha_1 \\ \alpha_2 \\ \vdots \\ \alpha_n \end{bmatrix} = \alpha \in \mathbb{R}^n$$

So each entry of Xu_1 is how far along u_1 each vector x is. So now, we can find the variance along u_1, the spread of the data along this vector.

$$Var(Xu_1) = \frac{1}{n-1} \|Xu_1\|^2$$

$$= \frac{1}{n-1} u_1^T X^T X u_1$$

$$= u_1^T C u_1$$

We want to maximise this, subject to the constraint $\|u_1\|^2 = u_1^T u_1 = 1$. A constrained optimisation problem. Using a Lagrange multiplier:

$$\mathcal{L}(u_1, \lambda) = u_1^T C u_1 - \lambda (u_1^T u_1 - 1)$$

$$\frac{\partial \mathcal{L}}{\partial u_1} = 2Cu_1 - 2\lambda u_1 = 0$$

$$Cu_1 = \lambda u_1$$

This is exactly the eigenvalue equation. So the direction u_1 that maximises variance is an eigenvector of the covariance matrix C. Additionally:

$$Var(Xu_1) = u_1^T C u_1$$

$$= u_1^T (\lambda u_1)$$

$$= \lambda u_1^T u_1$$

$$= \lambda$$

So to maximise variance, we choose the eigenvector corresponding to the largest eigenvalue λ_1. This is the first principle component. Following the same process, but with the constraint $u_2^T u_1 = 0$ (Second direction must be perp. to the first) We find that u_2 corresponds to the eigenvector with the second largest eigenvalue. In general, the k-th principal component is the eigenvector of C corresponding to the kth largest eigenvalue of C. The principal components are mutually orthogonal, they form an orthonormal basis for the subspace of maximum variance.

If we collect the top k eigenvectors as columns of a matrix $U_k \in \mathbb{R}^{d \times k}$, the projection of all data points into the k-dimensional subspace is:

$$Z = XU_k \in \mathbb{R}^{n \times k}$$

Thus, Z is the low-dimension representation. Each row $z^{(i)} = U_k^T x^{(i)}$ is the k-dimensional projection of data point i onto the principal component subspace.

Reconstruction Error

Given the k-dimensional projection $Z = XU_k$, we can reconstruct an approximation to the original by projecting back:

$$Z = XU_k$$

$$\hat{X} = ZU_k^T \quad (U_k \text{ is orthogonal, so its inverse is its transpose})$$

$\hat{X}$ is the best rank-k approximation to X, in the Column Space of U_k. The Reconstruction Error is the total squared difference between X and $\hat{X}$.

$$\|X - \hat{X}\|_F^2$$

Where $\|\cdot\|_F$ is the Frobenius Norm, the natural generalisation of the $L2$ norm to matrices. Square all the matrix elements, add them up and square root the result.

Variance Maximisation Equals Reconstruction Error Minimisation

As the title says, maximising the variance of the projections is identical to minimising the reconstruction error. This actually follows from the Pythagorean Theorem applied to orthogonal projections. For any unit vector u, the total variance of data decomposes as:

Total Variance = Variance along u

+

Reconstruction error from u

Since Total Variance is fixed, maximising variance along u is equivalent to minimising the reconstruction error.

Connection to SVD

The Singular Value Decomposition actually gives PCA directly. Recall that for any matrix X

$$X = U\Sigma V^T$$

where U, V are orthonormal, Σ is diagonal.

The Covariance Matrix is:

$$C = \frac{1}{n-1} X^T X$$

$$= \frac{1}{n-1} (U\Sigma V^T)^T (U\Sigma V^T)$$

$$= \frac{1}{n-1} V\Sigma^T U^T U \Sigma V^T$$

Note, since Σ is diagonal, $\Sigma^T = \Sigma$.
And since U is orthonormal, $U^T = U^{-1}$

$$= \frac{1}{n-1} V\Sigma^2 V^T$$

But see that this is exactly the Eigendecomposition of C!

$$\lambda_k = \frac{\sigma_k^2}{n-1}$$

This makes sense, since by construction, the vectors in V are the eigenvectors of X^TX.

So the PCA can be calculated in two equivalent ways:

- Eigendecomposition of C
  Form the $d \times d$ covariance matrix and find its eigenvectors

- SVD of X
  Compute $X = U\Sigma V^T$ and take the right singular vectors V.

In practice, the SVD approach is strongly preferred, as forming X^TX loses numerical precision.

Scores and Loadings

Some PCA Terminology:

- Loadings (Principal Components)
  The columns of V (right singular vectors of X, eigenvectors of C). These are the directions in feature space. Each loading vector has d components, one per original feature.

- Scores
  The projections $Z = XV_k = U_k\Sigma_k$
  These are the coordinates of each data point in the principal component space. Each score vector has k components, one per retained component

- Singular values / eigenvalues
  These measure the variance along each component. Larger means more important

Choosing Number of Components K

The fraction of total variance explained by the kth principal component is:

$$EVR_k = \frac{\lambda_k}{\sum_j \lambda_j}$$

For the cumulative explained variance ratio is where we sum up only the top k components

$$CEVR(k) = \frac{\sum_{j=1}^{k} \lambda_j}{\sum_j \lambda_j}$$

This quantity answers the question: "If I discard all but the top k components, what fraction of the data's total variation do I retain"?

A common rule of thumb is to choose k such that $CEVR(k) \geq 0.95$, retaining 95% of the total variance.